

# **SAM Paper Research**

2026.04.03

# Background

## SAM

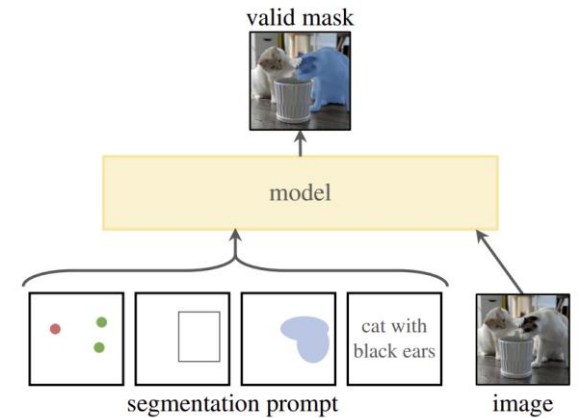
**Goal: To build a foundation model for image segmentation**

### Task

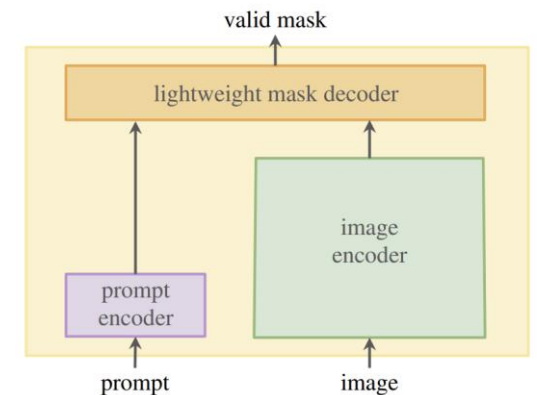
- Prompting 사용해서 zero-shot, few-shot 수행함
- 임의의 segmentation prompt가 주어졌을때 유효한 segmentation mask를 반환함
- Prompt는 이미지에서 무엇을 분할할지 지정하는 것으로 공간정보나 텍스트 정보를 포함함

### Model

- Image encoder → embedding
- Prompt encoder → embedding
- 두 정보 소스를 경량 mask decoder에서 결합하여 segmentation mask 예측함



(a) **Task:** promptable segmentation

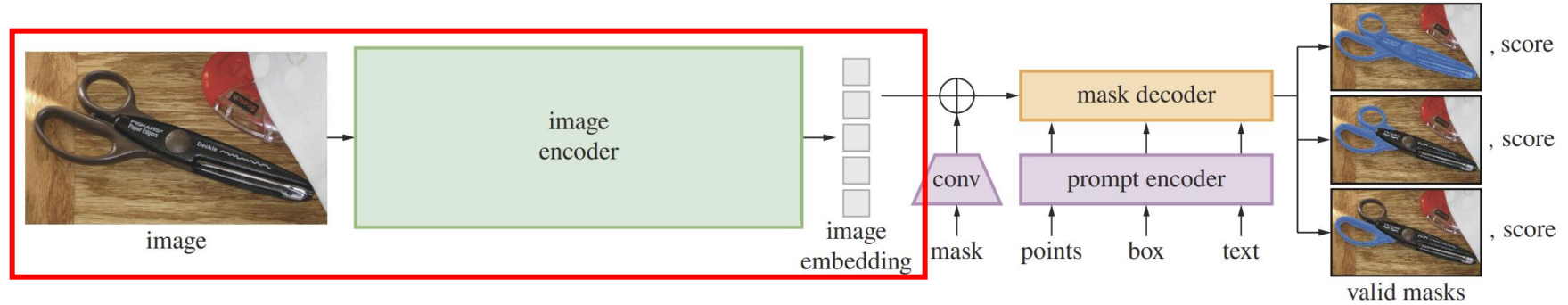


(b) **Model:** Segment Anything Model (SAM)

## Method

### Image Encoder

- Image (1024×1024)
- Patch Embedding (16×16)
  - Conv2D (kernel=16, stride=16)
  - 출력: (B, 64, 64, 768)
- Transformer Blocks (ViT)
  - Block(Attention (window or global), MLP, Residual connection)
  - 일부 block만 global attention이고 나머지는 window attention
- Conv Neck
  - (1×1 conv, 3×3 conv, LayerNorm) → (B, 256, 64, 64)
- image encoder는 이미지당 1회만 실행하고 이후 prompt마다 재사용함

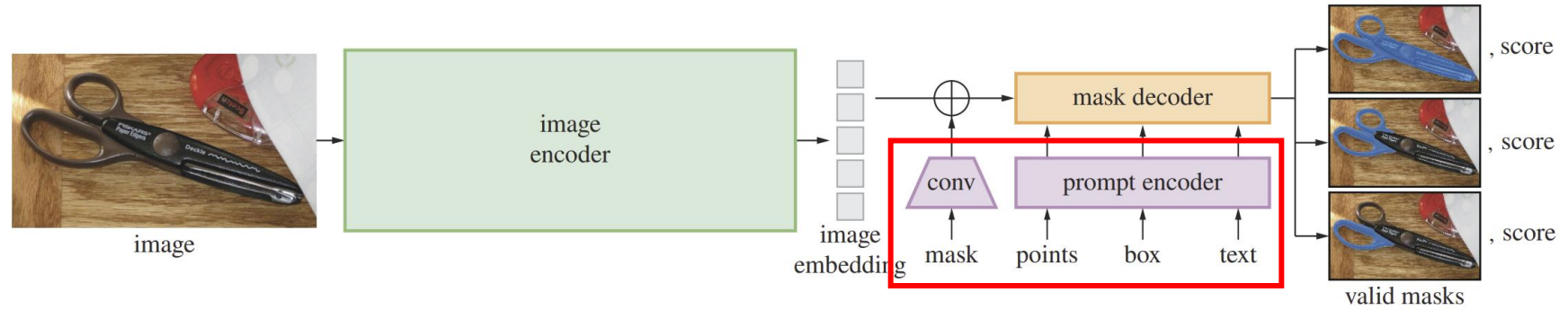


# SAM

## Method

### Prompt Encoder

- Sparse prompt (token 형태)
  - Point: 위치 encoding + FG/BG embedding
  - Box: (top-left, bottom-right) 각각 embedding
  - Text: CLIP encoder 사용  
→ vector token (256-dim)
- Dense Prompt (spatial feature)
  - 입력 mask → conv encoder로 embedding
  - Input 대비 1/16 resolution, 256-dim
  - image feature와 element-wise sum

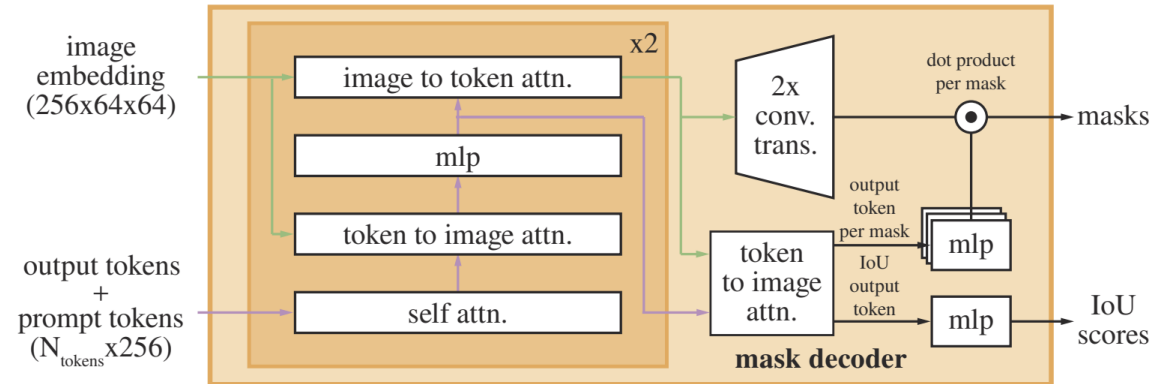
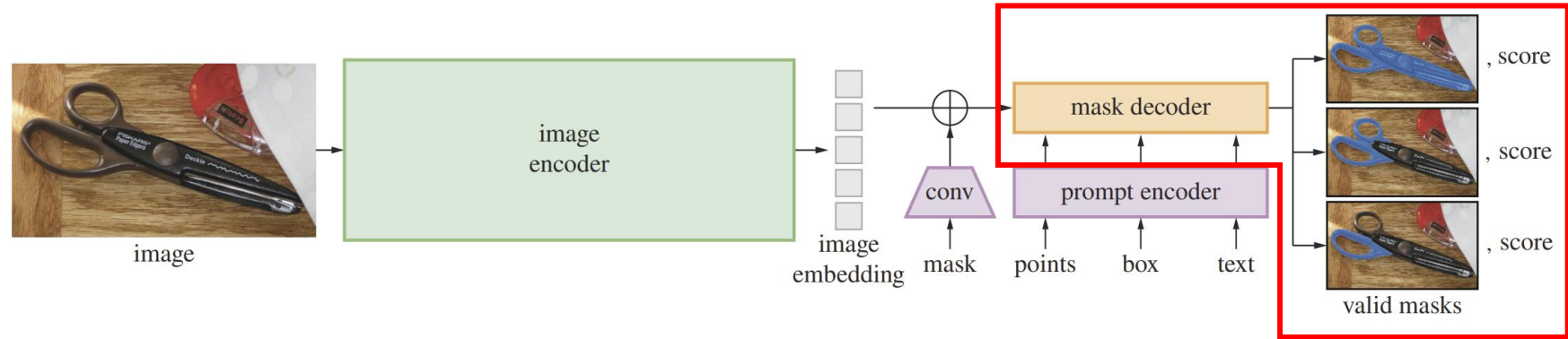


# SAM

## Method

### Mask Decoder

- Self-Attention (tokens)
  - prompt 간 관계 학습 (FG/BG 등 조건 정리)
- Cross-Attention (양방향)
  - Token → Image : 어디를 볼지 결정(region 선택)
  - Image → Token: image의 semantic 정보 반영
- Mask 생성
  - Image feature upsampling ( $\times 4$ )
  - 각 output token → 개별 MLP → mask vector 생성
  - $\text{mask} = (\text{mask vector}) \cdot (\text{image feature})$
- Output
  - Multiple masks (ambiguity 대응: part / whole 등), IoU score (mask quality)



# SAM2

## Task

**Goal: Image → Video로 확장된 Segmentation**

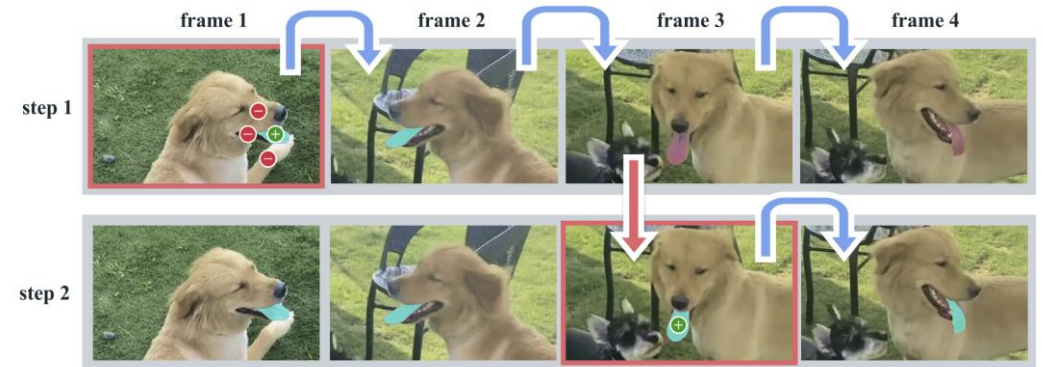
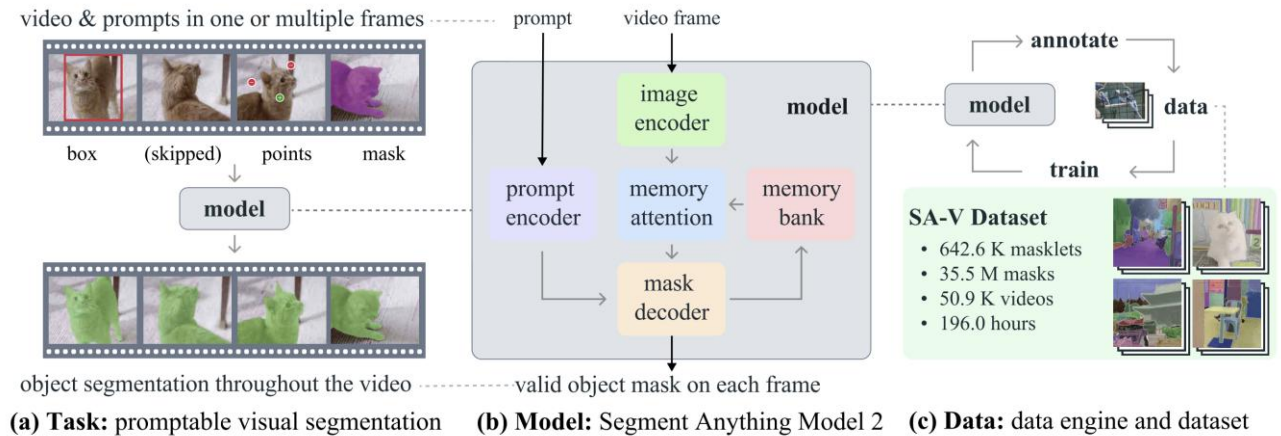
**Task** (Promptable Segmentation)

- Input
  - 비디오 프레임 sequence
  - prompt (point/ box/ mask, frame)
- Output
  - 모든 프레임에서의 object mask

→ Prompt 기반 객체 지정 후, 비디오 전 구간으로 확장

## Model

- Memory Bank: 이전 프레임의 feature + mask 정보 저장함
  - Memory Attention: 현재 프레임(Query), memory bank(Key, Value) → cross-attention 수행
- 과거 프레임 정보를 활용하여 현재 프레임에서 더 정확하고 일관된 segmentation



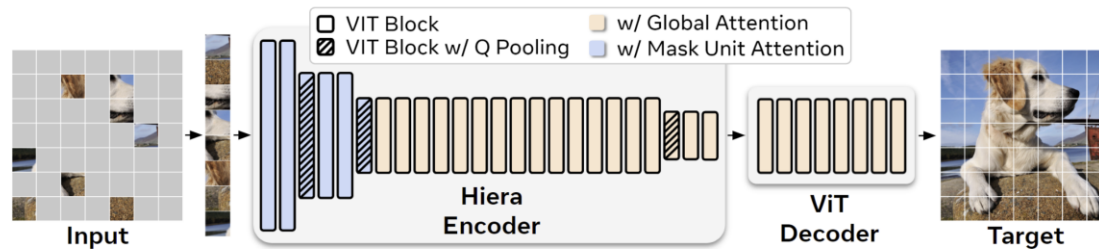
# SAM2

## Method

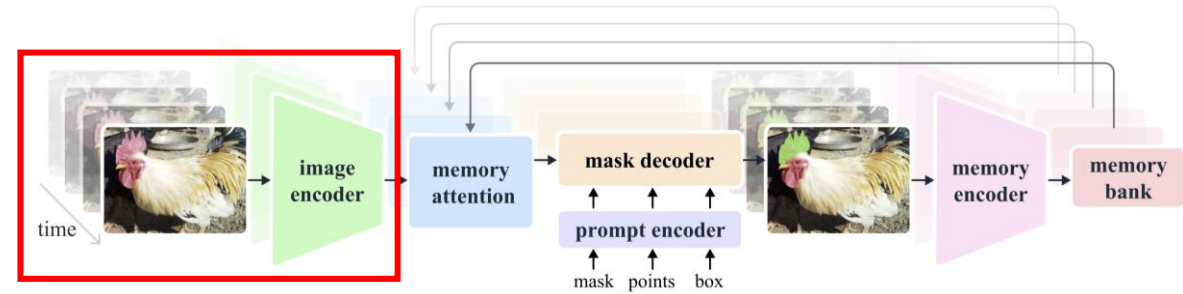
### Image Encoder

각 프레임을 feature embedding으로 변환함 → 이후 memory attention + mask decoder의 입력으로 주어짐

- Hierarchical Encoder (Hiera + MAE pretrain)
  - hierarchical 구조 → multi-scale feature 제공하여 프레임별로 object의 scale이 달라지는 video input에 적합함
  - stride 16, 32 (Stage 3, 4) → memory attention 입력 / semantic 정보 강함 → tracking에 중요
  - stride 4, 8 (Stage 1, 2) → mask decoder up-sampling에 활용 / detail 정보 → segmentation quality에 중요



- Streaming 방식 (Video 최적화)
  - 비디오를 frame 단위로 sequential 처리하고 image encoder는 각 프레임에 대해 한 번만 실행함
- 일부 layer에만 global attention 적용함



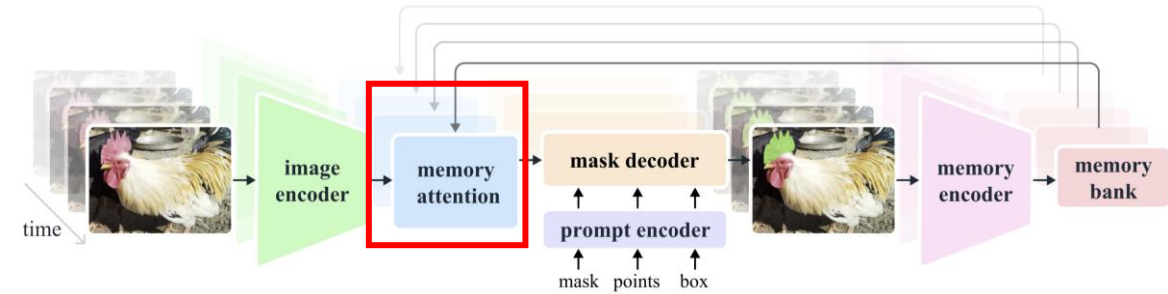


## Method

### Memory Attention

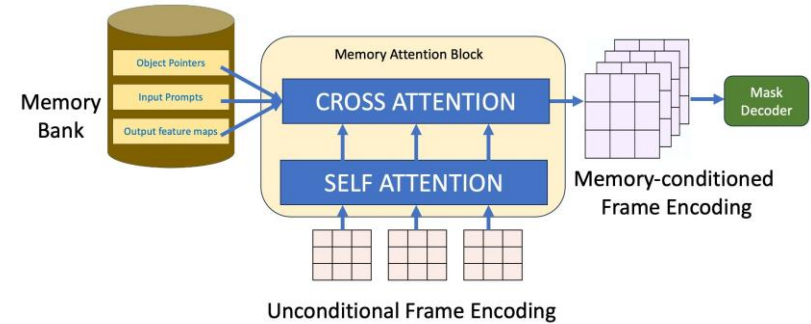
현재 프레임 feature를 과거 memory로 보정(conditioning) → 비디오에서 일관된 segmentation 유지

- Input: 현재 frame feature, memory bank(과거 프레임 feature, mask, object pointer tokens)
  - Transformer 기반 구조
    - L개의 transformer block (기본 L = 4)
    - Self-Attention: 현재 프레임 내부 관계 학습
    - Cross-Attention
      - 현재 프레임 feature = Query
      - memory bank = Key, Value
      - Q,K 연산으로 어떤 과거 프레임을 참고할지 결정하고 V를 통해 과거 정보를 가져와서 현재 프레임에 반영함
- 현재 프레임을 과거 프레임의 기억으로 보정하는 attention 구조



### MEMORY ATTENTION

An attention network that contextualizes frame embeddings with memory



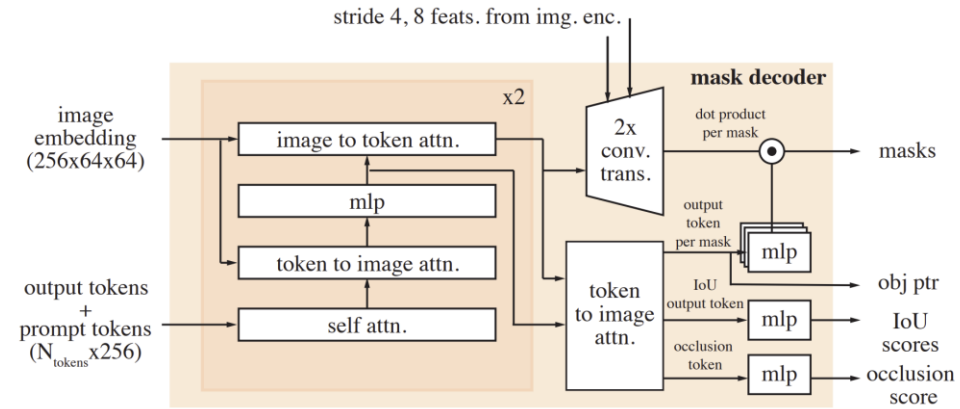
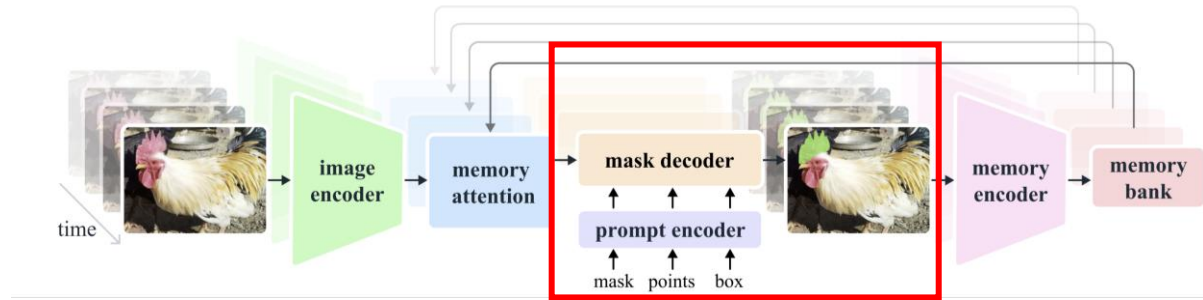


## Method

### Prompt Encoder (SAM과 동일한 구조)

### Mask Decoder

- 양방향 cross-attention, multiple mask prediction은 SAM과 동일함
- Object Pointer Token
  - Mask output을 pointer token으로 변환하여 memory bank에 저장함  
→ object identity 유지 (tracking)
- Occlusion Handling
  - occlusion prediction head 추가하여 객체가 현재 frame에 존재하는지 예측함
  - occluded frame인 경우 별도 embedding 추가하여 가려진 상황에서도 tracking 유지함
- High-resolution Skip Connection
  - image encoder의 low-level feature를 memory attention을 거치지 않고 decoder로 직접 전달함 → fine detail 복원



## Method

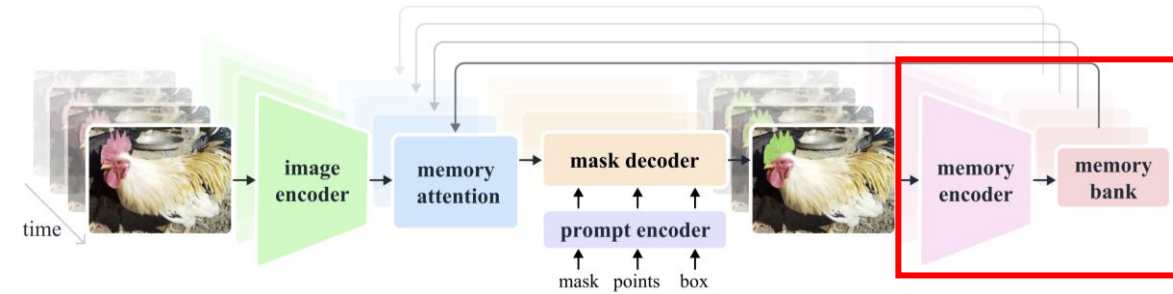
### Memory Encoder (현재 프레임의 예측 결과를 memory로 변환)

- Image encoder feature와 down-sampling(conv)된 output mask를 element-wise 결합
- Strong representation 유지하는 spatial feature map

### Memory Bank (과거 프레임 정보를 저장하는 공간)

- Prompt Memory: prompt가 들어간 frame 별도 저장
  - 초기 frame 정보 유지 → tracking anchor 역할 (최대 M개 유지)
- Spatial memory: 과거 프레임 feature + mask 정보
  - FIFO queue (최근 N 프레임 유지)
- Object Pointer: object의 semantic 요약 vector
  - object identity 유지 (tracking 안정성)

→ 현재 프레임 feature를 self-attention으로 정리한 뒤, memory와 cross-attention으로 결합해 보정된 feature를 만들



## Task

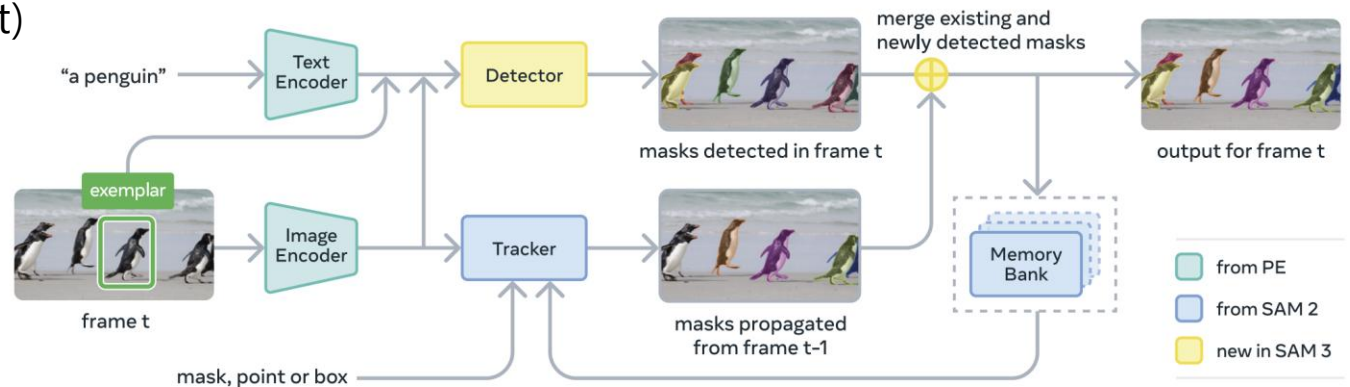
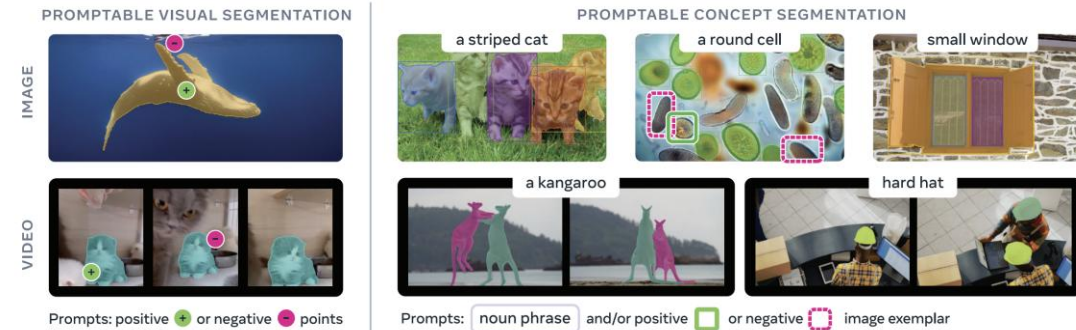
### Goal: Promptable Concept Segmentation

#### Task

- Prompt를 기반으로 특정 개념(concept)에 해당하는 모든 객체를 이미지 및 영상에서 탐지·분할·추적
- 개념 단위 segmentation + multi-object tracking

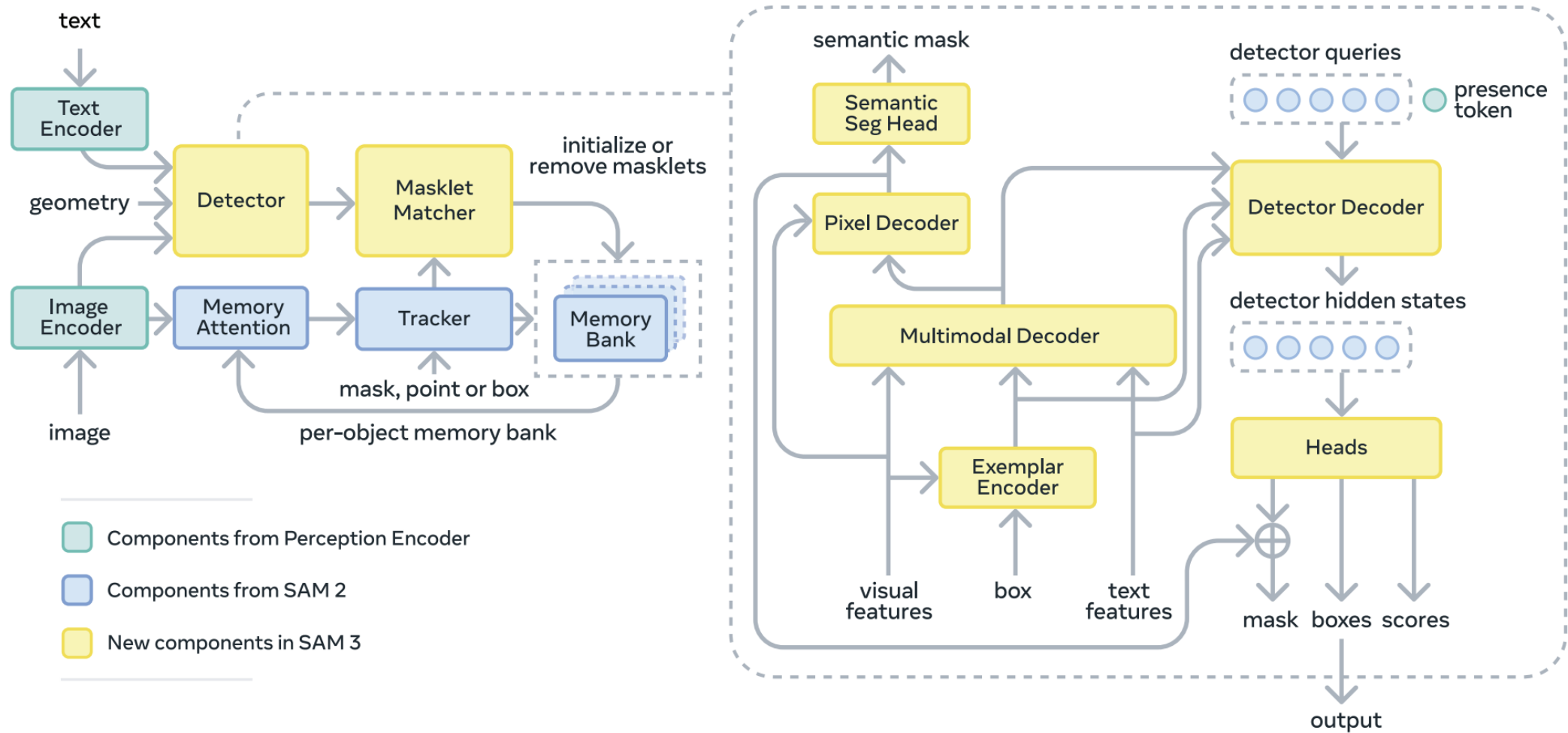
#### Model

- Concept-level segmentation (instance → concept)
- Detection + Tracking 분리 구조
  - Detector: 개념 인식 + 위치 탐지
  - Tracker: 시간 축에서 mask propagation
- Global presence token (Recognition 분리)
- Multimodal prompting
- SAM2 기반 memory tracking 확장



# SAM3

## Method



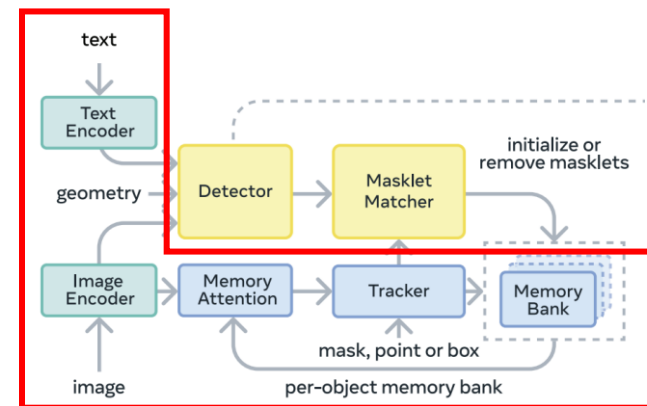
## Method

### Image and Text Encoder

- Perception Encoder (PE) 기반
- Visual feature와 text embedding 생성
- Image encoder의 output은 detector와 tracker의 공통 input으로 주어짐

### Memory Bank, Attn, Tracker (SAM2와 구조 유사)

- 각 object마다 과거 frame 정보가 저장됨(과거 appearance, conditioning frame, spatial memory)
- 현재 frame feature가 memory bank를 참고하여 현재 object가 어디로 갔는지 추정함 (self-attn + cross-attn)
- tracker는 이전 frame masklet을 현재 frame으로 propagation ( $M_{t-1} \rightarrow \hat{M}_t$ )



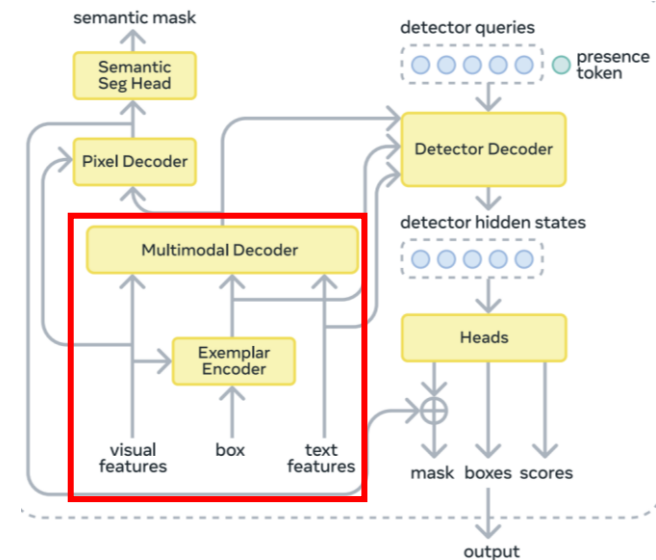
## Method

### Exemplar Encoder (optional)

- 위치 정보, positive / negative label, ROI pooled visual feature  
→ 합쳐서 exemplar token 생성함

### Multimodal Decoder

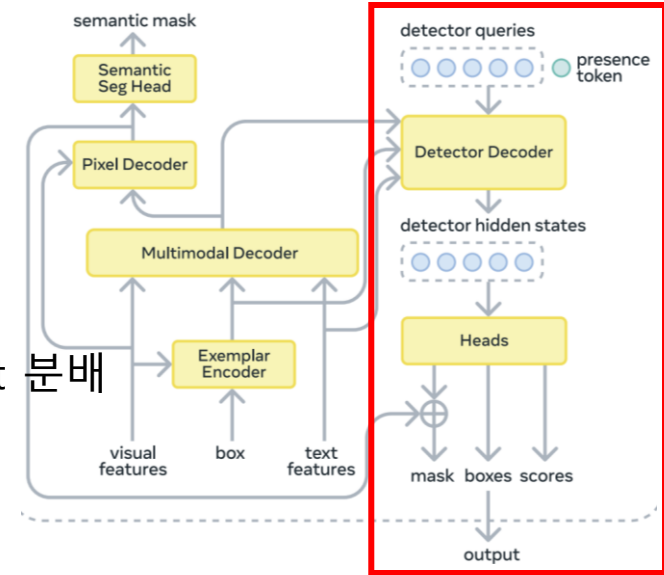
- 이미지 feature를 "무엇을 찾으라는 prompt"에 맞게 변형함
    - visual feature(unconditioned) from image encoder
    - prompt tokens(text feature + exemplar feature)
- >cross-attn> = conditioned image feature



## Method

### Detector Decoder (DETR)

- 현재 frame에서 prompt에 맞는 객체 후보들을 뽑음
  - learned object queries(default=200) self-attn to each other → query들끼리 object 분배
  - Cross-attn to the prompts tokens → 무엇을 찾는지 이해
  - Cross-attn to conditioned frame embeddings → 어디 있는지 찾기
  - MLP 거쳐서 box / score / mask를 예측함



### Presence Token: decouple the recognition and localization

- Query의 두가지 역할:
  - Recognition → global context 고려해야함
  - Localization → fine한 local information 중요함 } conflicts
- Presence token 따로 분리해서 concept이 image 안에 존재하는지 파악하고 query는 localization에만 집중할 수 있음
  - $p(query_i \text{ matches } NP) = p(query_i \text{ matches } NP \mid NP \text{ appears in image}) \cdot p(NP \text{ appears in image})$



# Analysis

## SAM3 - Text prompt

### Try 1 (segmentation test for an image)

- Normal vs damaged 두가지 프롬프트 적용
  - 모델이 물체 자체를 인식하지 못한다는 문제 발견
  - Text prompt로 cls\_name 자체를 주고 class별 object segmentation 성공 비율을 분석
  - Capsule, leather, pill, toothbrush, screw → 100% 인식함
  - Cable, carpet, hazelnut, tile → failure 많음
- class별 편차가 큼

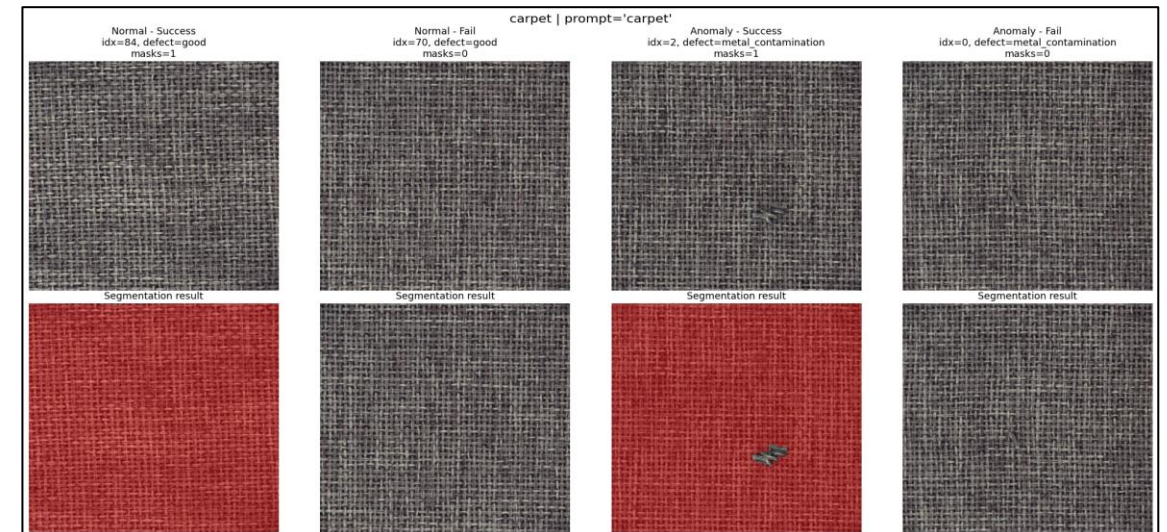
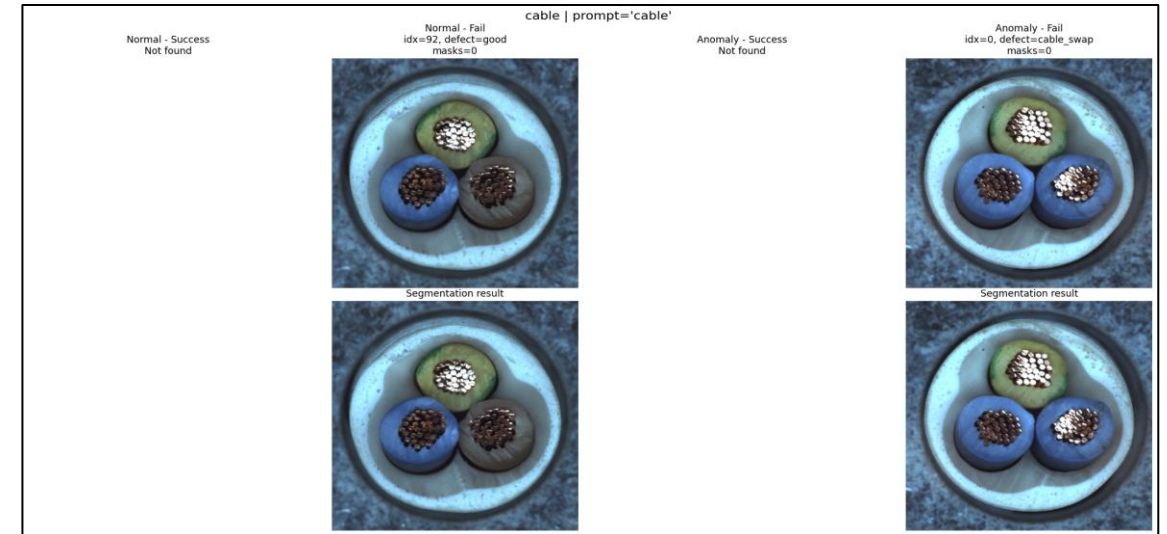
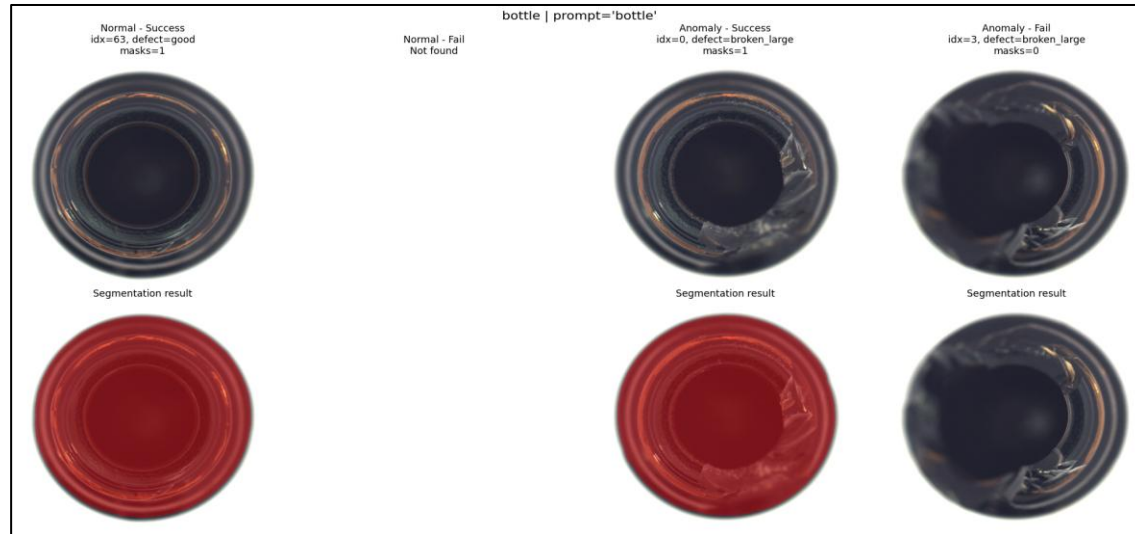
```
out1, out2 = inspect_two_prompts(
    processor,
    image,
    prompt1="normal bottle",
    prompt2="damaged bottle"
)
```

```
[normal bottle]
scores: tensor([], device='cuda:0', dtype=torch.bfloat16)
boxes: tensor([], device='cuda:0', size=(0, 4))
num_masks: 0
```

```
[damaged bottle]
scores: tensor([], device='cuda:0', dtype=torch.bfloat16)
boxes: tensor([], device='cuda:0', size=(0, 4))
num_masks: 0
```

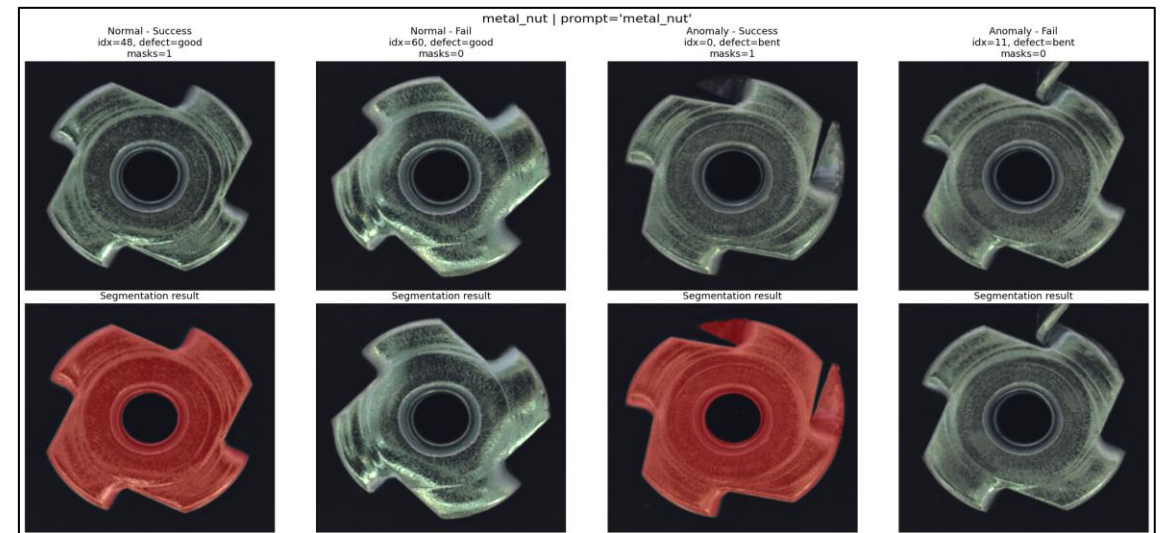
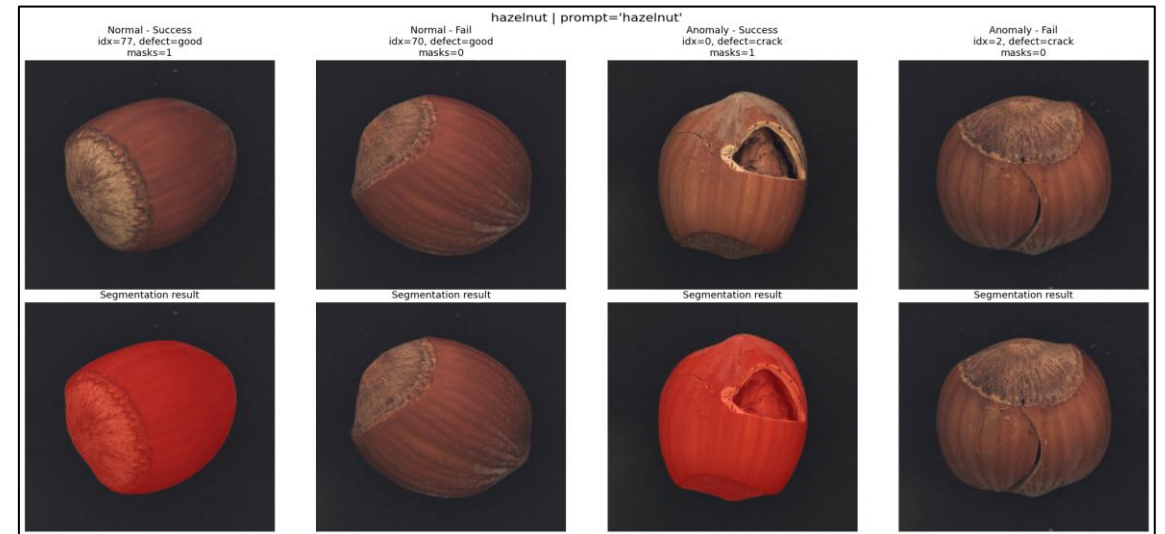
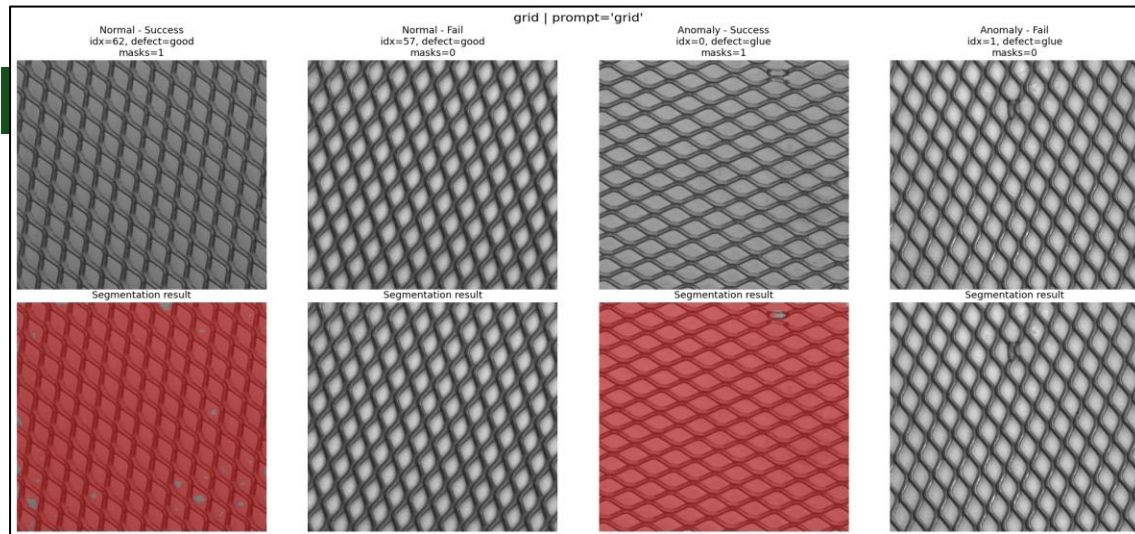
| Class    | Total | Success | Success Rate | Normal Rate | Anomaly Rate | metal_nut  | 115 | 77  | 66.96%  | 95.45%  | 60.22%  |
|----------|-------|---------|--------------|-------------|--------------|------------|-----|-----|---------|---------|---------|
| bottle   | 83    | 72      | 86.75%       | 100.00%     | 82.54%       | pill       | 167 | 167 | 100.00% | 100.00% | 100.00% |
| cable    | 150   | 0       | 0.00%        | 0.00%       | 0.00%        | screw      | 160 | 160 | 100.00% | 100.00% | 100.00% |
| capsule  | 132   | 132     | 100.00%      | 100.00%     | 100.00%      | tile       | 117 | 42  | 35.90%  | 42.42%  | 33.33%  |
| carpet   | 117   | 9       | 7.69%        | 3.57%       | 8.99%        | toothbrush | 42  | 42  | 100.00% | 100.00% | 100.00% |
| grid     | 78    | 50      | 64.10%       | 38.10%      | 73.68%       | transistor | 100 | 97  | 97.00%  | 100.00% | 92.50%  |
| hazelnut | 110   | 17      | 15.45%       | 2.50%       | 22.86%       | wood       | 79  | 77  | 97.47%  | 100.00% | 96.67%  |
| leather  | 124   | 124     | 100.00%      | 100.00%     | 100.00%      | zipper     | 151 | 125 | 82.78%  | 93.75%  | 79.83%  |

# Analysis

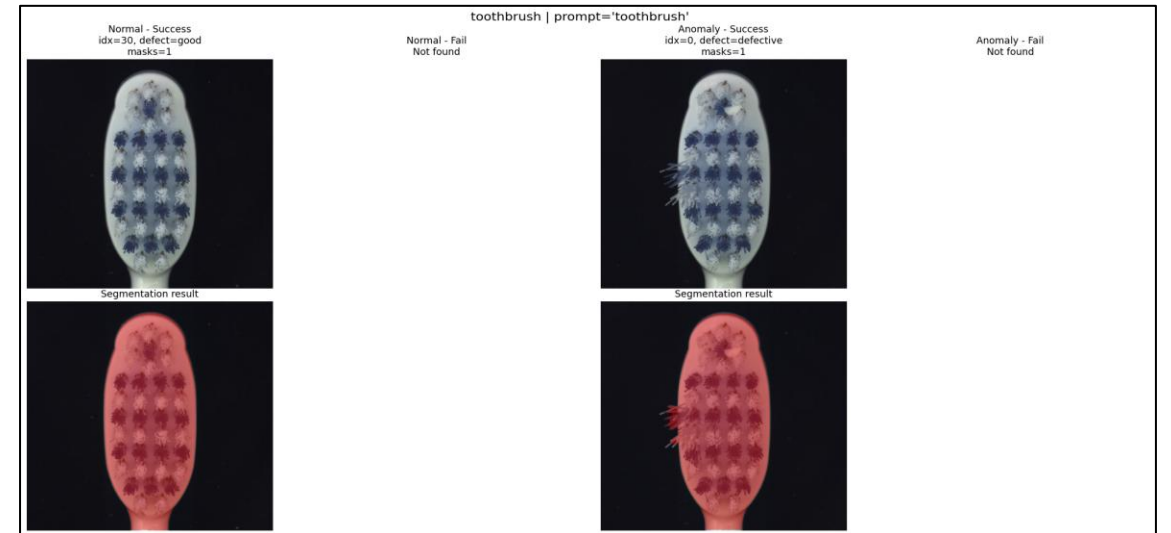
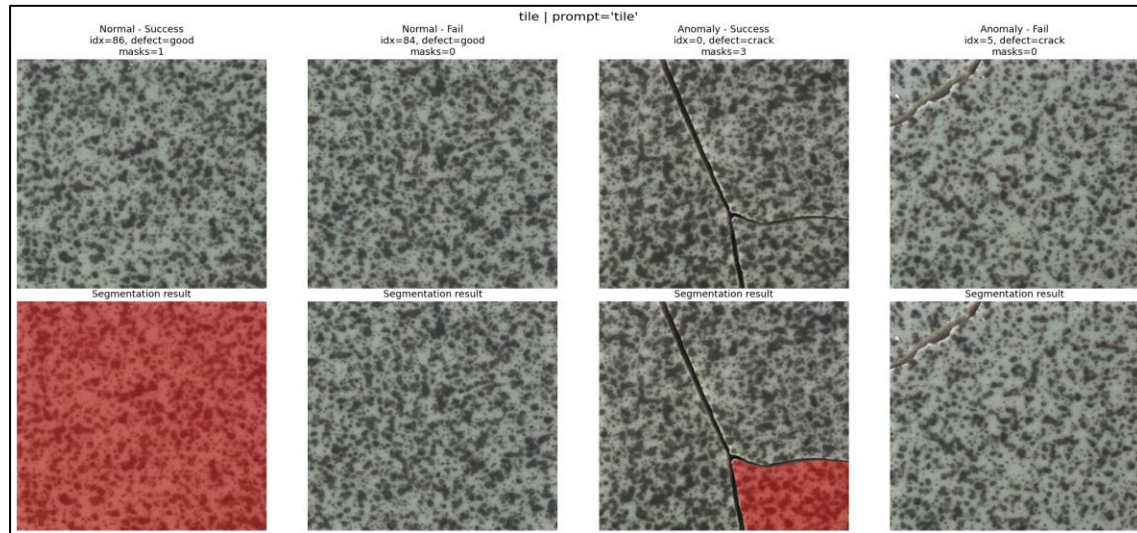
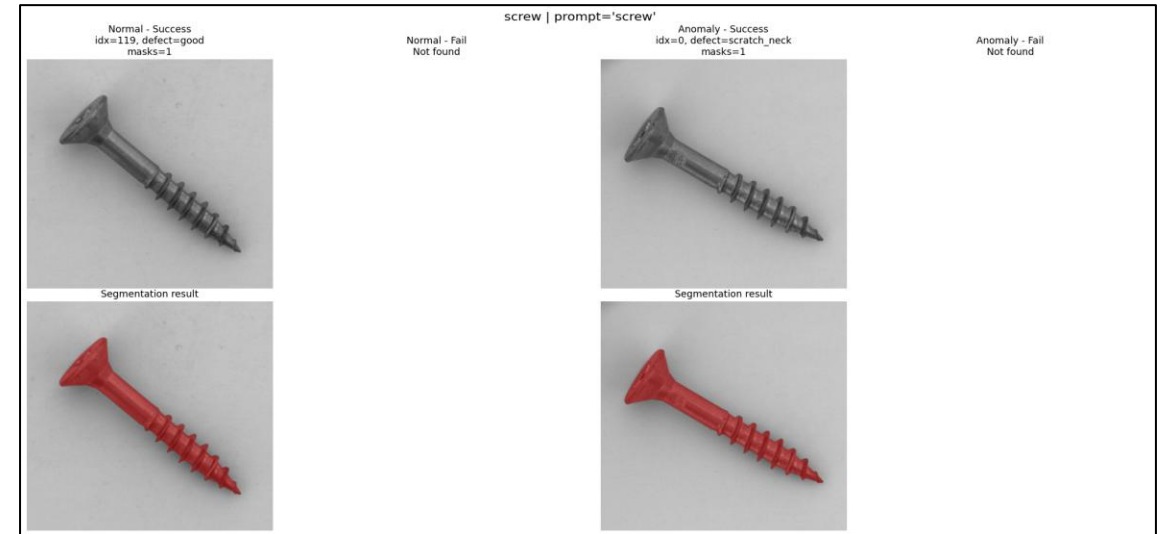
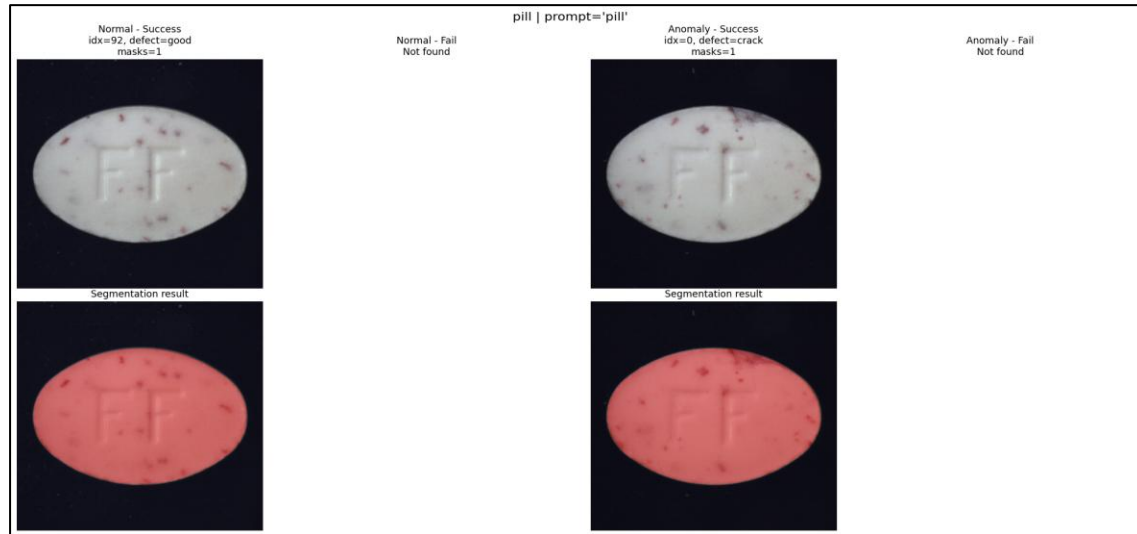




# Analysis

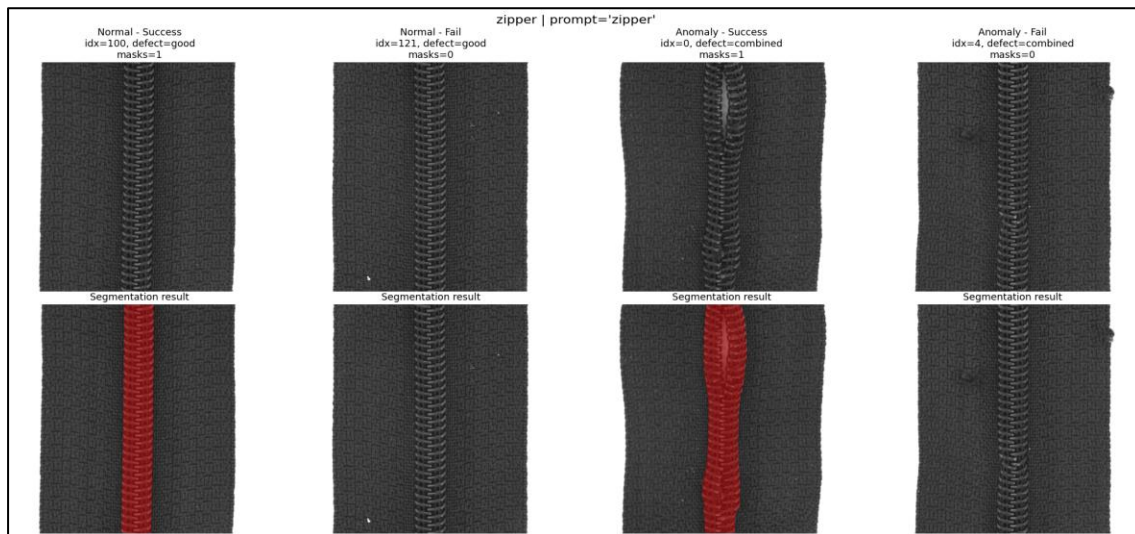
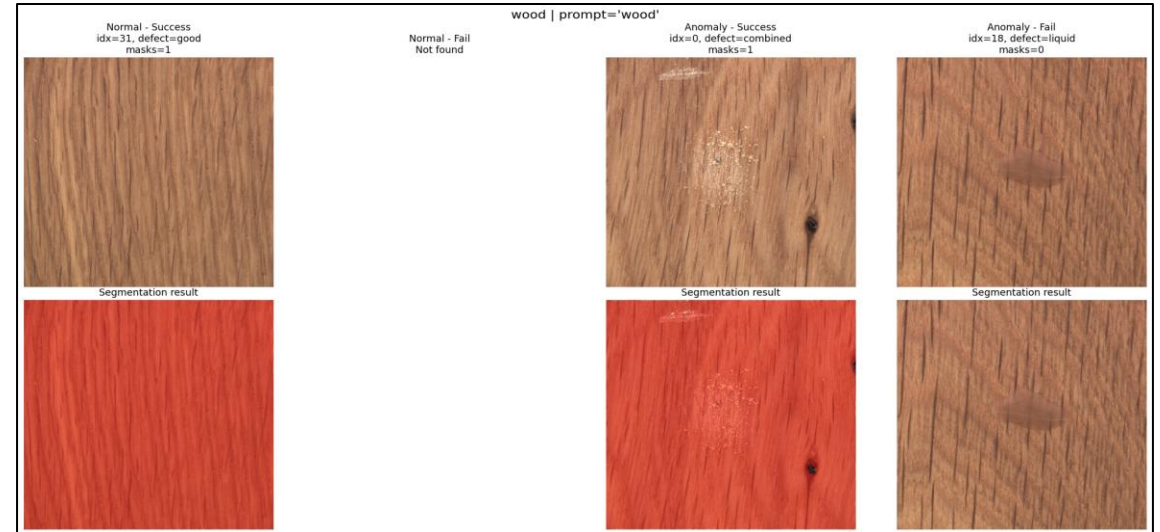
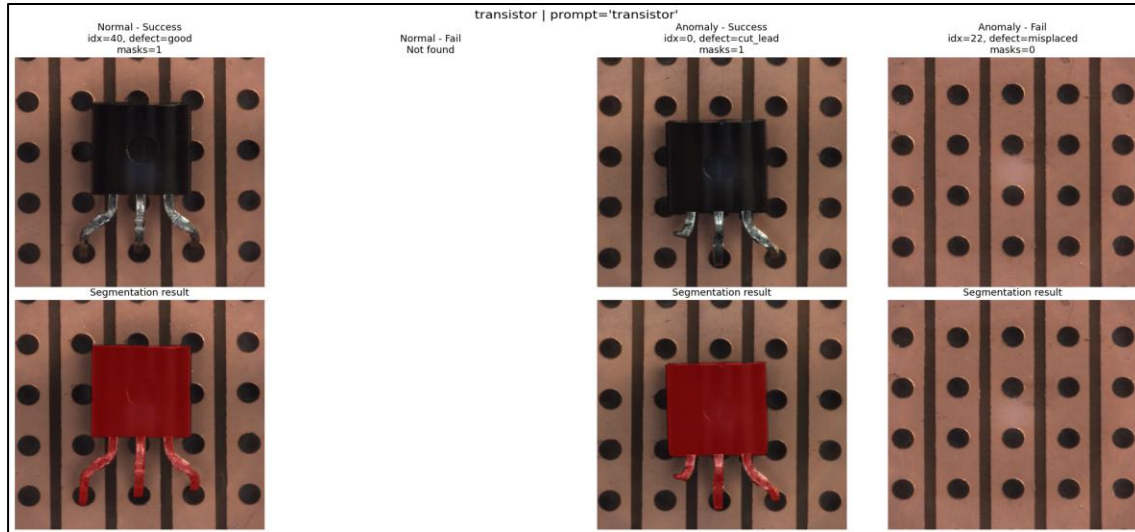


# Analysis





# Analysis



- mvTec dataset 자체가 이미 배경이 제거된 상태이고 물체의 확대된 사진이어서 일반적인 object 형상과 거리가 있음
- 대부분 클래스에서 anomaly 인식률이 normal보다 낮음
  - misplaced, 형태변화 등의 영향 있을 것으로 추정
- 드물지만 anomaly 부분만 segmentation 안된 예시 있음  
→ text 이외의 프롬프트 사용해보자!

# Analysis

## SAM3 - Text prompt + additional prompt

### Try 2 (improving localization)

- Train dataset의 normal에서 얻은 mask를 이용하여 box prompt 생성함
- 기존의 text prompt에 추가로 prior exemplar로 box를 제공함
- Try1의 결과와 비교하여 box prompt의 영향을 분석함
- Cable 제외한 모든 class에서 object 인식함

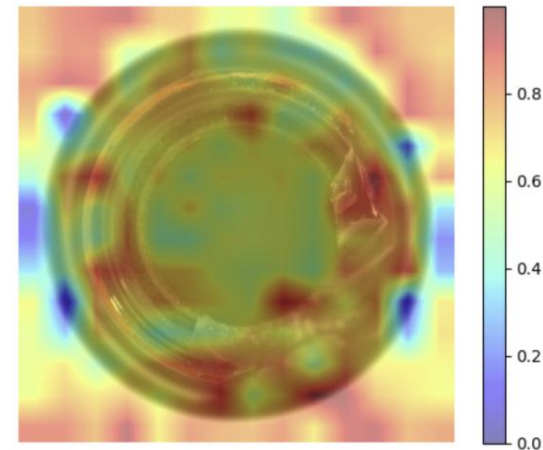
| Class    | Total | Success | Success Rate | Normal (Correct/Total) | Anomaly (Correct/Total) | metal_nut  | 115 | 115 | 100.00% | 22/22<br>(100.00%) | 93/93<br>(100.00%)   |
|----------|-------|---------|--------------|------------------------|-------------------------|------------|-----|-----|---------|--------------------|----------------------|
| bottle   | 83    | 83      | 100.00%      | 20/20<br>(100.00%)     | 63/63<br>(100.00%)      | pill       | 167 | 167 | 100.00% | 26/26<br>(100.00%) | 141/141<br>(100.00%) |
| cable    | 0     | 0       | 0.00%        | 0/0 (0.00%)            | 0/0 (0.00%)             | screw      | 160 | 160 | 100.00% | 41/41<br>(100.00%) | 119/119<br>(100.00%) |
| capsule  | 132   | 132     | 100.00%      | 23/23<br>(100.00%)     | 109/109<br>(100.00%)    | tile       | 117 | 117 | 100.00% | 33/33<br>(100.00%) | 84/84<br>(100.00%)   |
| carpet   | 117   | 117     | 100.00%      | 28/28<br>(100.00%)     | 89/89<br>(100.00%)      | toothbrush | 42  | 42  | 100.00% | 12/12<br>(100.00%) | 30/30<br>(100.00%)   |
| grid     | 78    | 78      | 100.00%      | 21/21<br>(100.00%)     | 57/57<br>(100.00%)      | transistor | 100 | 100 | 100.00% | 60/60<br>(100.00%) | 40/40<br>(100.00%)   |
| hazelnut | 110   | 110     | 100.00%      | 40/40<br>(100.00%)     | 70/70<br>(100.00%)      | wood       | 79  | 79  | 100.00% | 19/19<br>(100.00%) | 60/60<br>(100.00%)   |
| leather  | 124   | 124     | 100.00%      | 32/32<br>(100.00%)     | 92/92<br>(100.00%)      | zipper     | 151 | 151 | 100.00% | 32/32<br>(100.00%) | 119/119<br>(100.00%) |

# Analysis

## SAM3 – anomaly segmentation 수행

### Try 3 (Prototype Feature & Memory Bank 기반 Anomaly Segmentation)

- Motivation
  - 결함 유형이 다양하여 단일 representation으로는 한계 존재 → 결함 유형별 정보를 명시적으로 활용 필요
- Prototype Feature 생성
  - 각 결함 유형마다 (이미지 1장 + GT mask 1장) 사용함
  - GT mask 영역만 활용하여 feature 추출하여 결함 영역의 prototype feature 생성함
- Memory Bank 구축
  - Memory Bank = {class → defect type → prototype feature}
- Similarity Map (Anomaly Heatmap)
  - Query 이미지 → patch feature map 추출 (ViT 기반)
  - 각 patch와 prototype 간 cosine similarity 계산

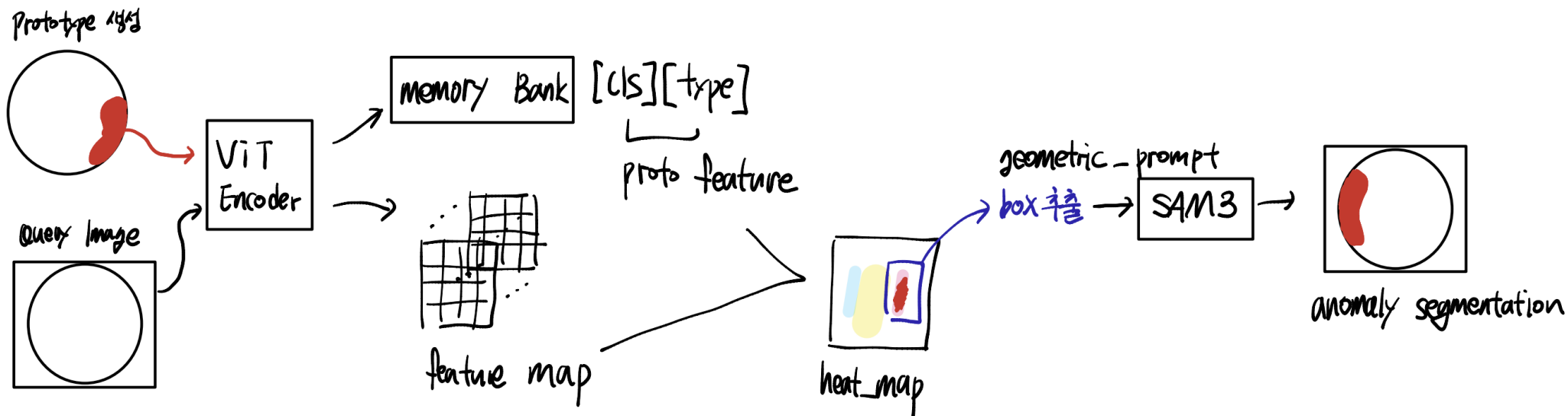
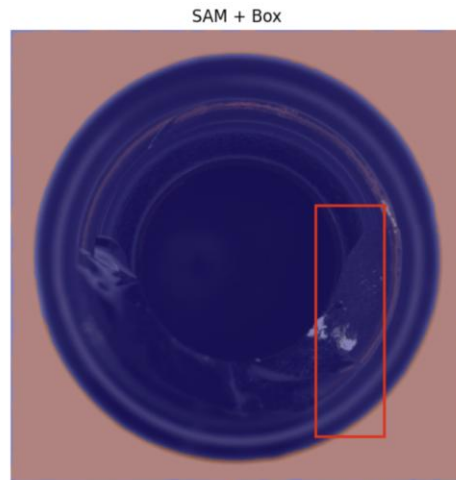


# Analysis

## SAM3 – anomaly segmentation 수행

### Try 3 (Prototype Feature & Memory Bank 기반 Anomaly Segmentation)

- Anomaly Localization → Segmentation
  - Similarity Map → 상위 영역(top-k) 선택 → bounding box 생성함
  - Bounding box를 SAM에 prompt로 제공하여 최종 anomaly segmentation mask 생성함





ELLab

